PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

Instituto de Ingeniería Biológica y Médica

iPre de Invierno: Desarrollo de una Power Distribution Board para prótesis de pierna robótica

PROFESORA: María Rodríguez

Vicente Maximiliano Corvalán Hernández
vcorvalh@uc.cl

30 de septiembre de 2025

1. Propósito y alcance

Este documento registra *cómo* se implementó el diseño de la PDB 12V para Prótesis Robótica y provee una guía práctica para su uso, réplica y extensión por estudiantes o ingenieros. Se enfoca en arquitectura, conexiones, consideraciones de ensamblaje, verificación, pruebas y solución de fallas. No profundiza en la teoría de los convertidores.

2. Resumen del sistema

- **Entrada:** Pack de baterías 12V (conector con llave).
- **Distribución:** LTC3780 (buck-boost) para estabilizar 12V; derivación a 5V mediante LM2596 (buck 5 V) para Raspberry Pi y lógica.
- **Actuación:** BTS7960 (puente H) para motor 12V (señales `RPWM/LPWM`, `R_EN/L_EN`).
- **Emergencia:** SRD-05VDC para disminuir inductancia en DC Link; 1N4007 en bobina (flyback) y en línea 12V (bloqueo retorno).
- **Protecciones:** En botón de emergencia para evitar *peaks* de voltaje por la inductancia y la alta corriente.
- **Sensado angular:** AS5600 (I^2C) por I^2C (no es cubierto en la elaboración de este informe).

3. Especificaciones eléctricas (referenciales)

Parámetro	**Símbolo**	**Valor**	**Observación**
Rango entrada BAT	V_{BAT}	$10-12.6\,\mathrm{V}$	Pack de baterías Makita
Rail regulado	$V_{12\mathrm{V}}$	$12.0\,\mathrm{V}$	LTC3780, buck-boost
Ondulación 12 V	$\Delta V_{12\mathrm{V}}$	$< 200\,\mathrm{mV}_{pp}$	Objetivo de validación
Rail lógica	$V_{5\mathrm{V}}$	$5.0\,\mathrm{V}$	LM2596, salida a Raspberry
Ondulación 5 V	$\Delta V_{5\mathrm{V}}$	$< 50\,\mathrm{mV}_{pp}$	Con $470\,\mu\mathrm{F}$ local
Corriente de prueba motor	I_{MOT}	$1\,\mathrm{A}$	Ensayo continuo
Interfaz control puente H	`EN/PWM`	—	Pull-down en entradas

Cuadro 1: Especificaciones eléctricas de referencia del sistema PDB

4. Conectores y pinout

4.1. Alimentación y distribución

Conector	Pins	Asignación	Observación
CN1	2	1: +12V 2: GND	Input de alimentación del PDB
CN2	2	1: 12V 2: GND	Salida rail regulado a potencia para motor drive
CN3	3	1: LPWM 2: L_EN 3: L_IS	Pines de control puente H
CN4	3	1: RPWM 2: R_EN 3: R_IS	Pines de control puente H
CN5,6,7,8	2	1:+5V 2:GND	Pines de alimentación para lógica
CN9	2	1:BTN_IN 2:BTN_OUT	Botón de emergencia y control de relé

Cuadro 2: Conectores de alimentación y distribución

4.2. Control de puente H (BTS7960 (puente H))

Conector	Pins	Asignación	Observación
J-HCTRL	6	1:RPWM 2:LPWM 3:R_EN 4: L_EN 5: 5V 6: GND	Entradas con pull-down/up
J-MOTOR	2	1: M+ 2: M-	Salida a motor 12 V

Cuadro 3: Interfaz de control y potencia del puente H

5. Procedimiento de encendido (*bring-up*)

1. **Energizar BAT** y verificar encendido de LEDs "on"de módulos LTC3780 y LM2596.
2. **Verificar rail 12 V** (salida LTC3780 (buck-boost)). Verificar que UVLO no esté activado, en caso de error, ajustar potenciómetro más cercano a la entrada de la batería.
3. **Habilitar 5 V** (LM2596 (buck 5 V)). Verificar 5.0V $\pm$ 5 y rizado $< 50_{pp}$ con 470 µF.
4. **Conectar carga lógica** (Raspberry). Confirmar arranque sin *brown-out*.
5. **Conectar puente H y motor**. Probar `EN` bajos (estado seguro), luego PWM a baja frecuencia/duty.
6. **Escalar a 1 A** de carga en motor; registrar rizado, caída de tensión y temperatura de módulos.
7. **Activar botón de emergencia** y comprobar que no hay alimentación en CN2.

6. Pruebas de validación

Eléctricas

- **Rizado y caídas:** medir en 12 V y 5 V con diferente carga de amperaje.
- **Transitorios:** arranques/inversión de dirección; observar estabilidad de `EN/PWM`.
- **Relé:** conmutación y supresión de flyback (1N4007).

Térmicas

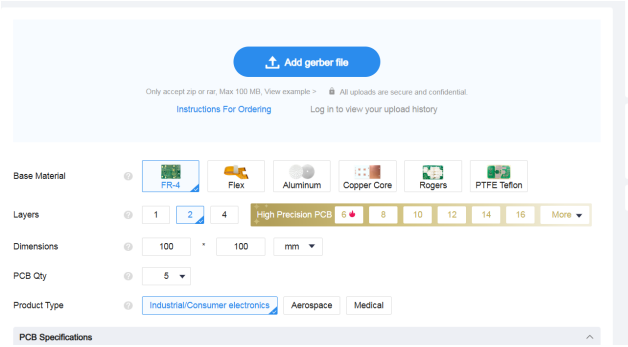
- **Monitor** de temperatura sobretodo en BTS7960 (puente H) en caso de haber cambiado motor a alguno que requiera mayor amperaje.

7. Recomendaciones para futuros diseños (layout y cableado)

- Separar **retornos** de alta corriente (motor) y control; unir en estrella cerca de BAT.
- Considerar las **corrientes de retorno** para evitar cross-talk en los voltajes de control del puente H.
- Minimizar **bucles** de conmutación; planos anchos en 12 V y GND; 470 µF o superior lo más próximo a la salida.
- Tener cuidado con el calor al soldar/desoldar componentes, pues pueden haber componentes antiguos en los módulos implementados que pueden tener problemas con altas temperaturas.

8. Instrucciones JLCPCB

Para enviar la PCB a fabricar a China en JLCPCB se ir a `https://jlcpcb.com` y clickear en "Order Now". Luego, se debe agregar los archivos Gerber como se muestra a continuación:

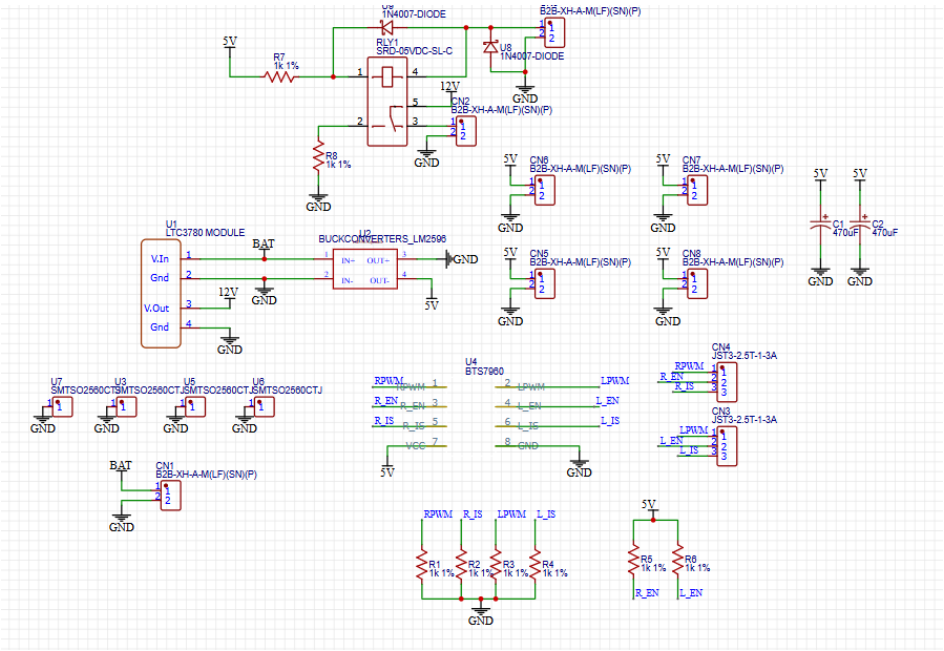


Subir el archivo Gerber y continuar con la compra agregando al carro. Poner ojo al método de envío ya que puede haber uno más barato en el mismo tiempo. Finalmente, revisar el carro y dar click en “Secure Checkout”.

En caso de haber algún inconveniente con la PCB se les contactará a través del correo de la cuenta creada en JLCPCB.

9. Anexos

9.1. Esquemático de la PCB diseñada



9.2. Layout de la PCB diseñada

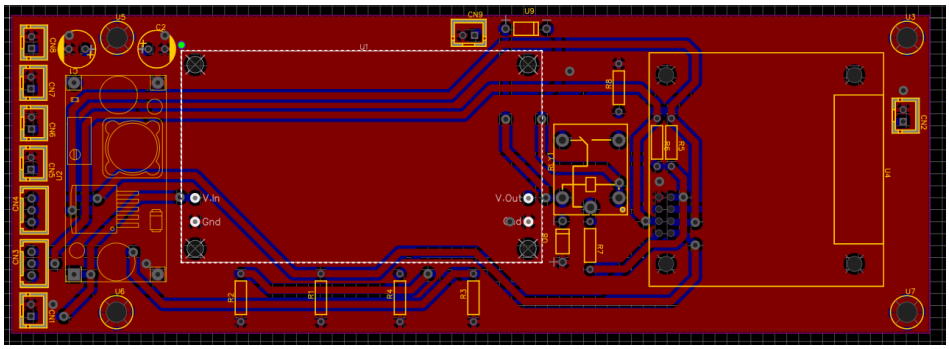


Figura 1: Top

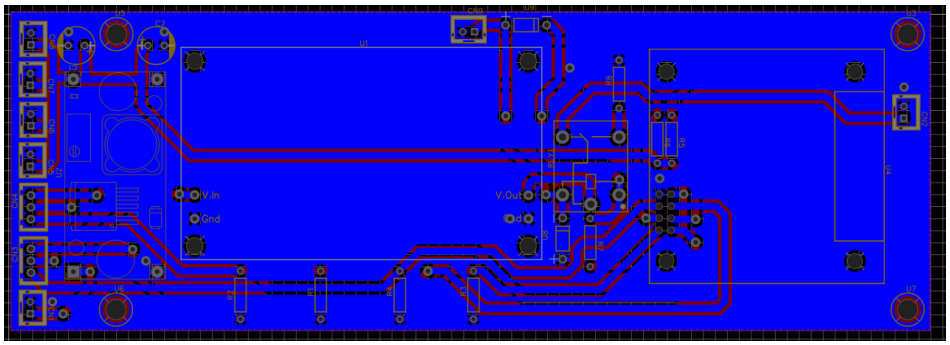


Figura 2: Bottom

9.3. Código simple de prueba (Arduino/ESP32)

```
/* ESP32 + BTS7960 (RPWM / LPWM) sin EN pins
   Barrido de velocidad: sube y baja antes de invertir el giro
   Rango velocidad: -255 a 255
*/

#define R_PWM 25    // RPWM
#define L_PWM 26    // LPWM

const int chR  = 0;
const int chL  = 1;
const int freq = 20000; // 20 kHz
const int res  = 8;     // 8 bits (0-255)

// Par metros del perfil de velocidad
const int VMAX      = 255; // pico de PWM
const int STEP      = 1;   // incremento/decremento por paso
const int STEP_TIME = 100; // ms entre pasos

void setup() {
  ledcSetup(chR, freq, res);
  ledcSetup(chL, freq, res);
  ledcAttachPin(R_PWM, chR);
  ledcAttachPin(L_PWM, chL);
}

void loop() {
  // Adelante: sube y baja
  rampTo(0,  VMAX,  STEP);    // 0      +VMAX
  rampTo(VMAX, 0,  -STEP);    // +VMAX      0
```

```
  delay(300);                    // pausa breve

  // Reversa: sube y baja (negativo)
  rampTo(0, -VMAX, -STEP);     // 0      -VMAX
  rampTo(-VMAX, 0,  STEP);     // -VMAX      0

  delay(300);
}

// ---------------- Funciones ----------------

void setMotor(int spd) {
  spd = constrain(spd, -255, 255);

  if (spd > 0) {                    // adelante
    ledcWrite(chL, 0);
    ledcWrite(chR, spd);
  } else if (spd < 0) {             // reversa
    ledcWrite(chR, 0);
    ledcWrite(chL, -spd);
  } else {                          // stop (coast)
    ledcWrite(chR, 0);
    ledcWrite(chL, 0);
  }
}

// Barrido lineal entre dos valores (incluye destino)
void rampTo(int fromVal, int toVal, int step) {
  if (step == 0) return;

  if ((step > 0 && fromVal > toVal) || (step < 0 && fromVal < toVal)) {
    step = -step; // corregir direcci n del paso si hace falta
  }

  for (int v = fromVal; (step > 0) ? v <= toVal : v >= toVal; v += step) {
    setMotor(v);
    delay(STEP_TIME);
  }
}
```

Notas de uso (resumen).

- PWM a **20 kHz** sobre ESP32 (`ledc*`); resolución **8 bits**. Ajustar `freq/res` según

ruido/linealidad.

- **Sin pines EN**: las entradas del BTS7960 deben tener *pull-down* para estado seguro al encender.
- Evita inversiones bruscas: el perfil hace rampa a 0 antes de invertir sentido.
- Verifica límites de corriente del motor y alimentación antes de subir `VMAX`.

9.4. Documentos

`https://uccl0-my.sharepoint.com/:f:/r/personal/marodriguezf_uc_cl/Documents/iPREs%20pr%C3%B3tesis/Vicente%202025?csf=1&web=1&e=6rEhAn`